Pointeri, referințe și transmiterea parametrilor – fișă

Oct 18, 2026 · @Mihnea-Andrei Teodorescu

1. Memoria pe scurt

Fiecare variabilă e o cutie în memorie, cu o adresă și o valoare. O referință nu e o cutie nouă, ci doar încă un nume pentru o cutie existentă. Un pointer e o cutie separată, în care stă o adresă.

```
int x = 5;
int &r = x;     // r = alt nume pentru cutia lui x
int *p = &x;    // p = cutie noua, care contine adresa lui x

x, r:  [ 5 ]                      <- o singura cutie, doua nume
p:     [adresa lui x] ---> x      <- cutie separata, care arata spre x
```

- `&x` = adresa lui x. `*p` = cutia de la adresa din p (aici, chiar x).
- `&` și `*` au două sensuri: într-o declarație (`int &r` , `int *p`) creează o referință / un pointer; într-o expresie (`&x` , `*p`) înseamnă „adresa lui” / „cutia de la adresa”.
- O referință se leagă o singură dată, la declarare, și nu se mută niciodată. `r = y;` copiază valoarea lui y în x.
- Un pointer se poate muta oricând: `p = &y;` .
- Variabilele locale stau pe stack și dispar automat la `}` -ul blocului lor.

2. Transmiterea parametrilor

Declarația funcției îți spune ce primește funcția, nu apelul: `f(x)` arată la fel și prin valoare, și prin referință.

Mod	Declarație	Apel	Ce primește funcția
prin valoare	`void f(int a)`	`f(x)`	o copie a valorii lui x (cutie nouă)
prin referință	`void f(int &r)`	`f(x)`	chiar cutia x, sub alt nume

Mod	Declarație	Apel	Ce primește funcția
prin pointer	`void f(int *c)`	`f(&x)` sau `f(p)`	o copie a adresei lui x (cutie nouă, aceeași adresă)
prin referință const	`void f(const int &r)`	`f(x)` sau `f(5)`	chiar x, dar fără voie să-l modifice

Cum arată memoria imediat după apel:

```
prin valoare:    main x: [5]        f a: [5]        <- doua cutii separate

prin referinta:  main x = f r: [5]                  <- o singura cutie

prin pointer:    main p: [adresa lui x] ---+
                                           +---> x: [5]   <- o singura cutie
x
                 f    c: [adresa lui x] ---+
```

- Dacă ai deja `int *p = &x;`, trimiți `f(p)`, fără `&`: p conține deja adresa.
- O referință din main (`int &rx = x;`) se trimite exact ca x: `f(rx)` e același lucru cu `f(x)`.
- `f(5)` nu merge la `int &r` (5 nu e o cutie), dar merge la `const int &r`.
- La `}`-ul funcției, parametrii (a, c) dispar. Variabilele din main rămân.
- Ca funcția să mute chiar pointerul din main, parametrul ar trebui să fie `int *&c` (referință la pointer). Fără `&`, c e o copie.

3. Regula pentru fiecare linie din funcție

Pentru fiecare linie din funcție te întrebi un singur lucru: se vede în main sau nu?

Parametrul e declarat	Linia	Se vede în main?	De ce
`int a`	`a = ...`	NU	a e o copie
`int &r`	`r = ...`	DA	r e chiar variabila din main

Parametrul e declarat	Linia	Se vede în main?	De ce
`int *c`	`c = ...`	NU	mută doar copia adresei
`int *c`	`*c = ...`	DA	scrie în cutia spre care pointează c în acel moment

- Un `*c` din dreapta (`... = *c + 1`) citește tot cutia spre care pointează c pe rândul curent, nu pe unul mai vechi.
- Imediat după apel, `*c` e chiar variabila din main spre care arăta pointerul trimis, nu o copie a ei.
- În main, `p = &y;` chiar mută p. Doar în funcție mutarea rămâne la copie.

4. Metoda de rezolvare

Faci un tabel cu o linie pentru fiecare instrucțiune și o coloană pentru fiecare variabilă și fiecare pointer, inclusiv pointerii din parametrii funcției.

1. **Coloanele.** Una pentru fiecare variabilă obișnuită. O referință stă în aceeași coloană cu variabila ei, de exemplu `y (= ry)`. Pentru fiecare pointer, o coloană `p →` în care scrii **numele** variabilei spre care pointează.
2. **Main până la apel.** Completezi rând cu rând.
3. **La apel.** Scrii ce e fiecare parametru: „a = copie a lui x (5)”, „r = y”, „c → x (aceeași adresă ca p)”.
4. **În funcție.** Aplici regula din secțiunea 3. La fiecare `*c`, citești coloana lui c de pe rândul curent.
5. **La `}`.** Coloanele parametrilor dispar. Coloanele din main rămân cum erau.
6. **Verificare.** `*p` din `cout` trebuie să fie egal cu valoarea variabilei din coloana lui p, pe ultimul rând. Dacă nu e, ai greșit undeva.

5. Exemplul 1 (ușor): copie, referință, pointer

Afișează `12 100 13 12`. Capcana e linia `r = y;`, care nu mută referința pe y.

```
#include <iostream>
using namespace std;

void f(int a, int &b, int *c) {
    a = a + 10;
    b = b + 10;
    *c = *c + 10;
}

int main() {
    int x = 1, y = 2, z = 3;
    int &r = x;
    r = y;
    y = 100;
    f(x, r, &z);
    cout << x << " " << y << " " << z << " " << r << endl;
    return 0;
}
```

Cod	x (= r)	y	z	c →	Ce se întâmplă
`int x = 1, y = 2, z = 3;`	1	2	3	—	
`int &r = x;`	1	2	3	—	r = alt nume pentru x, pentru totdeauna
`r = y;`	**2**	2	3	—	înseamnă `x = y`: copiază valoarea 2; r nu se mută pe y
`y = 100;`	2	**100**	3	—	se schimbă doar y
`f(x, r, &z);` intrare	2	100	3	z	a = copie a lui x (2); b = x; c = adresa lui z
`a = a + 10;`	2	100	3	z	NU: doar copia devine 12
`b = b + 10;`	**12**	100	3	z	DA: b e x
`*c = *c + 10;`	12	100	**13**	z	DA: c → z
`}` ieșire	12	100	13	—	a, b, c dispar

Memoria la intrarea în f:

```
main:  x = r = b: [2]      y: [100]      z: [3]
f:     a: [2]                            <- copie, cutie separata
       c: [adresa lui z] ---> z
```

6. Exemplul 2 (mediu): pointerul copiat care se mută

Afișează `5 9 10 9 10`. Capcanele: prima scriere `*c = ...` merge în z, iar `c = &b;` nu mută p.

```
#include <iostream>
using namespace std;

void g(int a, int &b, int *c) {
    *c = a * 2;
    c = &b;
    *c = *c + a;
    a = 0;
}

int main() {
    int x = 5, y = 8, z = 1;
    int &ry = y;
    int *p = &x;

    p = &z;
    ry = 4;

    g(x, y, p);
    cout << x << " " << y << " " << z << " " << ry << " " << *p << endl;
    return 0;
}
```

Cod	x	y (= ry = b)	z	p →	a	c →	Ce se întâmplă
`int x = 5, y = 8, z = 1;`	5	8	1	—	—	—	
`int &ry = y;`	5	8	1	—	—	—	ry = alt nume pentru y
`int *p = &x;`	5	8	1	x	—	—	
`p = &z;`	5	8	1	**z**	—	—	în main: p chiar se mută
`ry = 4;`	5	**4**	1	z	—	—	ry e y
`g(x, y, p);` intrare	5	4	1	z	**5**	**z**	a = copie a lui x; b = y; c = copie a lui p
`*c = a * 2;`	5	4	**10**	z	5	z	DA: c → z acum, deci z = 10

Cod	x	y (= ry = b)	z	p →	a	c →	Ce se întâmplă
`c = &b;`	5	4	10	z	5	**y**	NU: se mută doar copia
`*c = *c + a;`	5	**9**	10	z	5	y	DA: c → y, deci y = 4 + 5
`a = 0;`	5	9	10	z	**0**	y	NU: doar copia
`}` ieșire	5	9	10	z	—	—	a și c dispar; p n-a fost atins

Memoria după `c = &b;`:

```
main  p: [adresa lui z] ---> z: [10]
f     c: [adresa lui y] ---> y: [4]     <- doar c s-a mutat
```

7. Exemplul 3 (greu): aceeași funcție, apelată de două ori

Afișează `5 4 4`. La fiecare apel, parametrii se leagă din nou, iar la al doilea apel c și r sunt legate de alte variabile.

```
#include <iostream>
using namespace std;

void f(int *c, int &r) {
    r = *c;
    c = &r;
    *c = *c + 1;
}

int main() {
    int x = 3, y = 0;
    int *p = &x;
    f(p, y);
    p = &y;
    f(p, x);
    cout << x << " " << y << " " << *p << endl;
    return 0;
}
```

Cod	x	y	p →	c →	r e...	Ce se întâmplă
`int x = 3, y = 0;`	3	0	—	—	—	

Cod	x	y	p →	c →	r e…	Ce se întâmplă
`int *p = &x;`	3	0	x	—	—	
`f(p, y);` intrare	3	0	x	**x**	**y**	c = copie a lui p; r = y
`r = *c;`	3	**3**	x	x	y	DA: y = x
`c = &r;`	3	3	x	**y**	y	NU: doar copia
`*c = *c + 1;`	3	**4**	x	y	y	DA: y = 3 + 1
`}` ieșire	3	4	x	—	—	c și r dispar
`p = &y;`	3	4	**y**	—	—	în main: p chiar se mută
`f(p, x);` intrare	3	4	y	**y**	**x**	acum c → y și r = x
`r = *c;`	**4**	4	y	y	x	DA: x = y
`c = &r;`	4	4	y	**x**	x	NU: doar copia
`*c = *c + 1;`	**5**	4	y	x	x	DA: x = 4 + 1
`}` ieșire	5	4	y	—	—	

La `cout`: x = 5, y = 4, `*p` = y = 4.

8. Greșeli frecvente

Toate au apărut în exercițiile din 18 oct.; fiecare are o întrebare cu care o prinzi.

Greșeala	Ce e de fapt	Întrebarea de control
Tratezi referința ca pe o cutie separată (`ra` = 10, dar `a` = 20)	referința și variabila ei au mereu aceeași valoare	Sunt în aceeași coloană?
Crezi că `r = y;` mută r pe y	copiază valoarea lui y în variabila lui r	De ce variabilă s-a legat r la declarare?
Crezi că `c = ...` din funcție mută și p din main	mută doar copia	Linia începe cu `*`?
Crezi că la intrare c pointează spre o copie a lui x	se copiază doar adresa, deci `*c` e chiar x	Ce adresă a primit c la apel?
Citești un `*c` din dreapta de pe un rând vechi	`*c` = cutia spre care pointează c pe rândul curent	Spre ce pointează c acum?
Dai pentru `*p` o valoare pe care n-o are nicio variabilă	imposibil: `*p` e mereu o variabilă existentă	Ce scrie în coloana lui p?